使用 enumitem 宏包定制列表

Javier Bezos*

Version 3.11
2025/02/06

Contents

1 简介 **2**

2 快速参考 **2**

3 键 **4**
3.1 标签与交叉引用格式 4
3.2 标签的水平间距 6
3.3 编号、停止与续编 9
3.4 系列 9
3.5 惩罚值 10
3.6 注入代码 10
3.7 描述样式 11
3.8 紧凑列表 12
3.9 “Wide” lists 12
3.10 类似 enumerate 的标签 12
3.11 通用键与值 13

4 行内列表 **13**

5 全局设置 **14**

6 依赖字号的设置 **15**

7 克隆基本列表 **16**

8 计数器详解 **17**
8.1 新计数器表示 17
8.2 重启 enumerate 17

9 宏包选项 **18**

10 trivlist 问题 **18**

11 示例 **18**

12 后记 **20**
12.1 LaTeX 列表 20
12.2 已知问题 21
12.3 3.0 新特性 21
12.4 错误修复 21
12.5 致谢 21

*问题报告、评论和建议请发送至 http://www.texnia.com/enumitem.html。

注意 与 3.5 版相比的变更和新功能以 新特性 X.X 标出。最新功能可能仍不稳定。请将发现的问题报告至 https://github.com/jbezos/enumitem/issues，这比在邮件列表或网络论坛上抱怨更有帮助。欢迎提交 fork 和 pull request。

警告 3.6 版引入了两个新键：left 和 first。如果你的文档中定义了使用这些名称的系列，将会报错。只需重命名它们，或设置宏包选项 series=override。

1 简介

本宏包提供了你可能需要的绝大多数灵活性，用于定制三种基本列表环境（enumerate、itemize 和 description）以及设计自己的列表，采用 ⟨键⟩=⟨值⟩语法：

- 精美的标签和引用，包括与 enumerate 宏包类似的语法。
- 标签定位的替代方式，引入了新参数（labelindent）和类似制表位的设置（left）。
- 设置可全局应用，也可仅应用于三种列表类型之一，甚至单个列表（包括\topsep ）。
- 行内列表。
- 多种描述样式（同时修复了一些间距问题）。
- 正确格式化的 trivlist。
- 将多个列表组合为一个单元处理，以及计数器续编。

接口中使用了一种” 继承” 机制。你可以全局设置列表的行为，然后覆盖 enumerate 的若干参数，再在某个特定实例中覆盖少数参数。值将按层级逐级查找。

本宏包扩展了列表的语法，允许使用可选参数，其中提供了一组 key=value 形式的参数。这些键与众所周知的列表参数等价。请参阅 LaTeX 手册了解它们的说明。下一节解释 enumitem 提供的扩展。

- 垂直间距：
  - topsep
  - partopsep
  - parsep
  - itemsep
- 水平间距：
  - leftmargin
  - rightmargin
  - listparindent
  - labelwidth
  - labelsep
  - itemindent

示例 一个简单的例子：

```
\begin{itemize}[itemsep=1ex, leftmargin=1cm]
```

注意 查看这些参数如何工作的一个方法是使用 layouts 宏包（手册名为 layman.pdf）。参见图 1。

2 快速参考

一些常用设置。详见下文手册。

- 完全去除列表中的垂直间距：

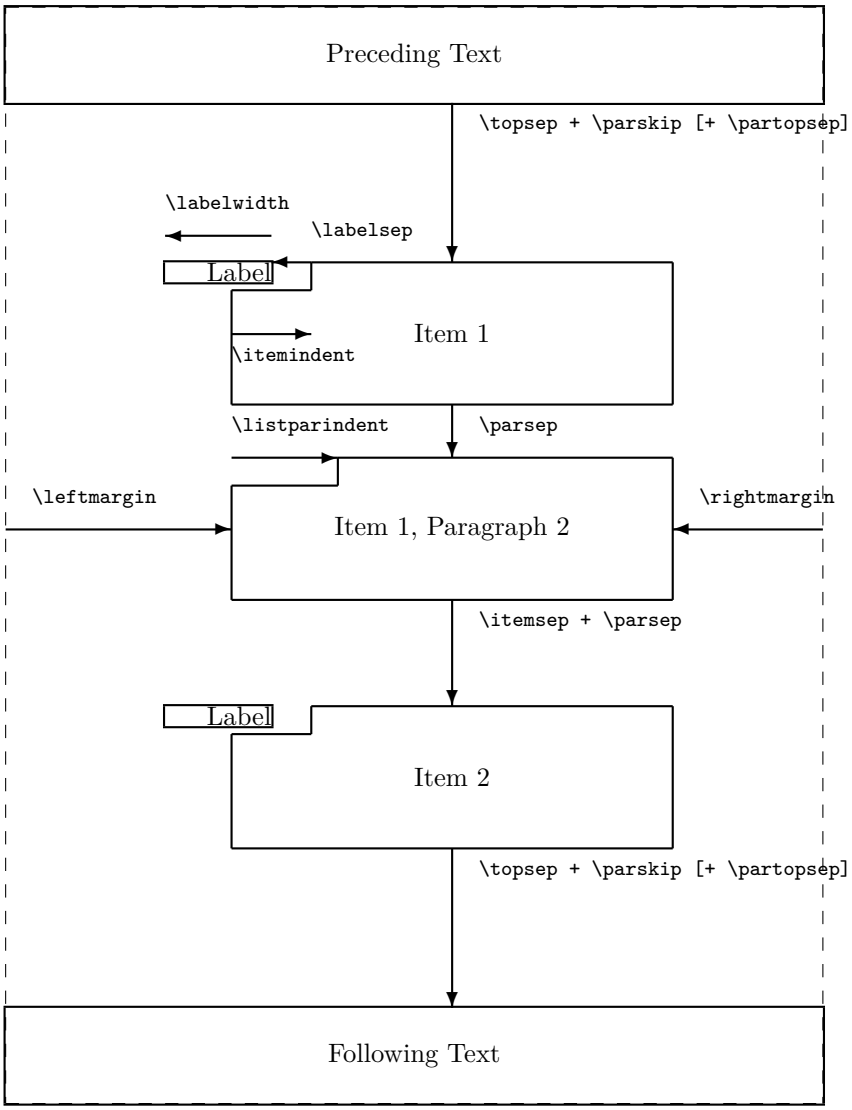


Figure 1: 列表参数

```
\begin{enumerate}[nosep]
```

- 完全去除所有列表中的垂直间距：

```
\setlist{nosep}
```

- 让标签从边距开始，条目文字从当前 parindent 开始：

```
\begin{enumerate}[left=0pt .. \parindent]
```

- 像 enumerate 那样配置标签：加载宏包选项 `shortlabels`，然后作为第一个元素写入你的 label：

```
\begin{enumerate}[(1)]
```

- 在”暂停”后继续上一个列表：

```
\begin{enumerate}[resume*]
```

- 行内使用三个基本列表：只需添加宏包选项 `inline`，然后使用环境 `enumerate*`、`itemize*` 和 `description*`。
- 设置带括号的数字标签，但交叉引用不带括号：

```
\begin{enumerate}[label=(\arabic*), ref=\arabic*]
```

3 键

本节介绍显示列表中的键。其中大多数也可用于行内列表，后者还有更多可用键（见 4）。

警告 如果值完全被花括号包围，它们会被剥离。如果你需要花括号，必须写成双份。这是 keyval 的默认行为，enumitem 只是模拟它。

3.1 标签与交叉引用格式

`label=`⟨*commands*⟩

设置当前层级使用的标签。一组不带参数的`\alph`、`\Alph`、`\arabic`、`\roman` 和`\Roman` 的星号版本代表 `enumerate` 中的当前计数器。[1] 它也适用于`\value`（前提是不需要计算最宽标签，或使用了 `widest*`，见下文）。

注意 如果你想像 enumerate 宏包那样设置标签，请使用”短标签”（见第 3.10 节）。

示例 以下代码输出 *a*)、*b*) 等（这是西班牙的标准样式，以前芝加哥也使用）：

```
\begin{enumerate}[label=\emph{\alph*})]
```

警告 `label` 的值是移动参数，脆弱命令必须保护，但计数器除外。因此，`\value` 的使用有些棘手，因为`\the` 或`\ifnum` 期望一个实际值，而在处理 `label` 以将内部的 `*` 替换为带计数器参数的形式时并非如此。最佳解决方案通常是借助`\AddEnumerateCounter` 将逻辑封装在一个新的”计数器”中。[2]

示例 一个更花哨的例子（看起来不美观，但仅用于展示可能性；需要 color 和 pifont）：

[1] 实际上，星号目前就是参数，但未来可能改变。请将它们视为星号变体并遵循相应语法。
[2] 诚然这有些复杂。更好的方法正在开发中。

```
\begin{enumerate}[label=\protect\fcolorbox{blue}{yellow}{\protect\ding{\value*}}]
```

`label*=`⟨*commands*⟩

类似 `label`，但其值追加到父标签之后。例如，以下定义了一个 `legal` 列表（1.、1.1.、1.1.1. 等）：

```
\newlist{legal}{enumerate}{10}
\setlist[legal]{label*=\arabic*.}
```

`ref=`⟨*commands*⟩

默认情况下，`label` 同时设置交叉引用和`\the...` 的形式（覆盖父层级中的设置），但你可以用此键定义不同的格式。例如，去掉右括号：

```
\begin{enumerate}[label=\emph{\alph*}), ref=\emph{\alph*}]
```

注意 在 `label` 和 `ref` 中，计数器都可以照常使用。因此，如果当前层级是第二级：

```
\begin{enumerate}[label=\theenumi.\arabic*.]
```

或

```
\begin{enumerate}[label=\arabic{enumi}.\arabic*.]
```

注意 `label` 不会累积形成引用。如果你想要，比如第一级为 1)、第二级为 a)，从而得到类似 $1.a$ 的结果，必须用 `ref` 设置。你可以使用`\ref{level1}.\ref{level2}` 配合适当的 `ref` 设置，但正如 Robin Fairbairns 在 TEX FAQ 中指出的：

> ……这样做既繁琐又容易出错。更重要的是，这并不可取，因为你构造了一个不灵活的视觉表示（你无法一次性改变列表元素的所有引用）。

这很有道理，我建议遵循这个建议，但有时你可能想要这样的效果：

```
... subitem \ref{level2} of item \ref{level1} ...
```

警告 `ref` 的值是移动参数，脆弱命令必须保护，但计数器除外。

`font=`⟨*commands*⟩　`format=`⟨*commands*⟩

设置标签字体。当通过`\item` 的可选参数更改标签时，以及在 `description` 中很有用。⟨*commands*⟩ 中最后一个命令可以接受条目标签作为参数。在 `description` 类设置生效时，你可能需要以`\normalfont` 开头。同义词是 `format`。实际上，此键可用于在每个`\item` 处、标签之前执行的任何内容。

`align=left`　`align=right`　`align=parleft`

标签如何对齐（相对于标签框边缘）。有三个可选值：`left`、默认的 `right` 和 `parleft`（一个宽度为`\labelwidth` 的 parbox，左对齐文本）。控制标签间距的参数应正确设置，手动设置或更方便地使用 `*` 设置（见下文）：

```
\begin{enumerate}[label=\Roman*., align=left, leftmargin=*]
```

当标签框应为其自然宽度时，使用 left。

`\SetLabelAlign{⟨value⟩}{⟨commands⟩}`

可以用\SetLabelAlign 定义新的对齐类型（或重新定义现有类型）；预定义值等价于：

```
\SetLabelAlign{right}{\hss\llap{#1}}
\SetLabelAlign{left}{#1\hfil}
\SetLabelAlign{parleft}{\strut\smash{\parbox[t]\labelwidth{\raggedright##1}}}
```

示例 虽然主要用于对齐，但此命令还有其他用途（例如提供的 parleft）。例如，以下代码将所有 align=right 的标签设为上标：

```
\SetLabelAlign{right}{\hss\llap{\textsuperscript{#1}}}
```

当然，也可以使用新名称。

注意 如果定义中的最后一项是 skip（通常是\hfil ），有时会被 description 移除。如果出于某种原因你想避免这种情况，只需在末尾加\null 。

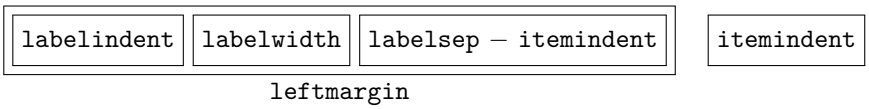
注意 如果你想让 align 和 font 的内部设置被忽略，可以在 before 中覆盖 enumitem 对\makelabel 的定义：

```
\begin{description}[before={\renewcommand\makelabel[1]{\ref{##1}}}]
```

或者，定义一个宏并用\let 。

3.2 标签的水平间距

当前层级左边距中的水平空间按以下方式分配：[3]



这里 labelindent 是 enumitem 引入的新参数，见下文。其余是标准 LaTeX 中的参数。

实际上，布局更复杂，因为标签框（即 labelwidth）可能伸入边距，这意味着 labelindent 取负值。

注意 由于\parindent 在列表内部不被直接使用，而是内部设为\itemindent 或\listparindent ，当作为参数值使用时，enumitem 返回全局值，即它在最外层列表之外的值。

注意 新特性 3.6 如果你觉得这些参数令人困惑，你不是一个人。你可以在第一个条目前（或在 first 中）写\DrawEnumitemLabel 来可视化它们，它从上到下画 4 条线：labelindent、labelwidth、labelsep、itemindent（正细负粗）；leftmargin 用两条竖线标记。

`labelindent=⟨length⟩`
`\labelindent`

此参数是 enumitem 新增的，表示从外层列表/文本边距到标签框左边缘的空白。这意味着存在冗余，因为其中一个参数依赖于其他参数，即它必须从其他值计算得出，如下所述。默认情况下，计算值是 labelindent，即使显式设置了某个值（默认为 0 pt）。因此，如果你将其设为某个值，很可能你想将某个其他参数设为! 或 *，否则它会被忽略。

新增了一个计数器长度\labelindent 。

这五个参数的关系如下：

$$\texttt{\textbackslash leftmargin} + \texttt{\textbackslash itemindent} = \texttt{\textbackslash labelindent} + \texttt{\textbackslash labelwidth} + \texttt{\textbackslash labelsep}$$

[3]诚然，这些图并不完全清晰，我打算在未来版本中改进。

`left=`⟨*labelindent*⟩
`left=`⟨*labelindent*⟩ `..` ⟨*leftmargin*⟩

新特性 3.6 这是一个便捷键，用于快速设置标签最常见的布局。你可以将其视为一种带有两个制表位的" 标尺": 标签起始位置和文本起始位置（两者都相对于正常边距)。仅给定 ⟨*labelindent*⟩ 时，左边距（即" 文本起始位置") 用 `labelsep` 计算。它内部使用 `widest`，因此关于 `description` 的说明也适用于此: 你可能需要更改计算出的参数（例如，配合 `align=left` 使用 `itemindent=*`)。

示例 典型设置:

```
\begin{enumerate}[left= 0pt]
\begin{enumerate}[left= 0pt .. \parindent]
\begin{enumerate}[left= \parindent]
\begin{enumerate}[left= \parindent .. 2\parindent]
\begin{enumerate}[left= -\parindent .. 0pt]
```

注意 标签宽度设为默认的最宽值。如果列表中有 ≥ 10 的阿拉伯数字，你可能还需要设置 `widest`。

注意 `left=`⟨*labelindent*⟩ 设置 `leftmargin=*`; `left=`⟨*labelindent*⟩ `..` ⟨*leftmargin*⟩ 设置 `labelsep=*`。

`leftmargin=!` `itemindent=!` `labelsep=!` `labelwidth=!` `labelindent=!`

设置哪个值由其他值计算得出。默认是 `labelindent=!`，但注意有些键会设置另一个值（`wide` 和 description 的 `style`)。计算在所有键都读完后进行。显式值不会丢失，因此对于以下层级设置:

```
leftmargin=2em
labelindent=1em,leftmargin=!
labelindent=!
```

`leftmargin` 再次为 2em，而 `labelindent` 是计算参数。

注意 在 `align=right`（默认）下，`labelindent=!` 和 `labelwidth=!` 在实际中表现相似。

`leftmargin=*` `itemindent=*` `labelsep=*` `labelwidth=*` `labelindent=*`

类似前面，但此外 `labelwidth` 首先设为当前标签的宽度，使用`\arabic*` 中 *0*、`\roman*` 中 *viiii*、`\alph*` 中 *m* 的默认值，大写形式类似（这些值可用 `widest` 更改，见下文)。示例:

```
\begin{itemize}[label=\textbullet, leftmargin=*]
\begin{enumerate}[label=\roman*), leftmargin=*, widest=iii]
\begin{itemize}[label      = \textbullet,
                leftmargin = 2pc,
                labelsep   = *]
\begin{enumerate}[label       = \arabic*.,
                  labelindent = \parindent,
                  leftmargin  = 2\parindent,
                  labelsep    = *]
```

最有用的是 `labelsep=*` 和 `leftmargin=*`。前者使条目正文从固定位置开始（即 `leftmargin`)，后者从取决于标签的可变位置开始（当然在一个列表内总是相同的)。

注意 遗憾的是，LaTeX 没有定义一个应用于所有列表的默认 `labelsep`——只是使用当前值。使用 enumitem 你可以为每个列表设置默认值，如下所述，因此如果你想确保 `labelsep` 在你的控制之下，只需:

```
\setlist{labelsep=.5em}
```

注意 `labelwidth=*` 和 `labelwidth=!` 是同义词。请谨慎使用，因为它们可能取负值，这没有意义（会显示警告)。

`widest=`⟨*string*⟩　`widest*=`⟨*integer*⟩　`widest`

如需与 `*`-值配合使用。它覆盖最宽打印计数器的默认值。有时，如果列表不太长，`\alph` 的值 `a` 比默认的 `m` 更合理：

```
\begin{enumerate}[leftmargin=*, widest=a] % Assume standard 2nd level
```

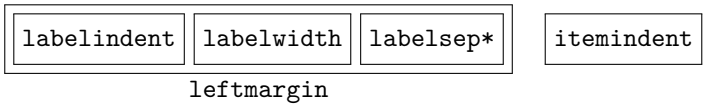
不带值时恢复默认。使用 `widest*` 时，字符串使用 ⟨*integer*⟩ 作为计数器值构建（例如，对于`\roman`，`widest=viii` 和 `widest*=8` 相同）。

由于`\value` 返回的不是字符串而是数字，`widest` 和 `*` 值不能与它一起使用。但 `widest*` 是数字，所以允许。

新特性 3.6 它也可用于 `itemize` 和 `description`。但由于后者在标签内部做了一些复杂的格式化，你可能需要更改计算出的参数（例如，配合 `align=left` 使用 `itemindent=*`）。

`labelsep*=`⟨*length*⟩

记住 `labelsep` 跨越 `leftmargin` 的一部分和 `itemindent`（如果后者非零）。这常常令人困惑，因此提供了一个新键——使用 `labelsep*` 时值从左边距计算（它只是设置`\labelsep` 然后加上`\itemindent`，但此外后续对 `itemindent` 的更改也会被考虑）：



`labelindent*=`⟨*length*⟩

类似 `labelindent`，但从当前列表的左边距计算，而不是从外层列表/文本的左边距计算。

示例 第一种模式使标签与周围的`\parindent` 对齐，同时条目正文根据标签和固定的 `labelsep` 缩进：

```
labelindent = \parindent,
leftmargin  = *
```

一个常见变体是使标签与周围文本对齐（记住如果 `labelindent` 不是计算参数，默认为 `0pt`）：

```
leftmargin = *
```

前者在第一层看起来更好，后者在后续层级更可取。可以轻松设置：

```
\setlist{leftmargin=*}
\setlist[1]{labelindent=\parindent} % Only the level 1
```

示例 第二种模式使条目正文与周围的`\parindent` 对齐。在这种情况下（记住如果未设置，`labelindent` 是计算参数）：

```
leftmargin = \parindent
```

示例 第三种模式是标签与`\parindent` 对齐，条目正文与 `2\parindent` 对齐：

```
labelindent = \parindent,
leftmargin  = 2\parindent,
itemsep     = *
```

同样，一个变体是标签与周围文本对齐，条目正文与`\parindent` 对齐：

```
leftmargin = \parindent,
itemsep    = *
```

3.3 编号、停止与续编

`start=`⟨*integer*⟩

设置第一个条目的编号。

`resume`

计数器从之前的 `enumerate` 继续，instead of being reset to 1.

```
\begin{enumerate}
\item First item.
\item Second item.
\end{enumerate}
Text.
\begin{enumerate}[resume]
\item Third item
\end{enumerate}
```

这是局部完成的。如果你想要全局续编，参见下一节关于 series.

`resume*`

类似 `resume`，但同时使用之前列表的选项。此选项必须限制在环境的可选参数中（这是它唯一有意义的地方）。应谨慎使用——如果你经常使用它，很可能你需要定义一个新列表（见 7）。允许其他键，在这种情况下保存的选项被当前列表中的选项覆盖（即 `resume*` 的位置无关紧要）。如果某个列表系列使用 `resume*`，选项取自第一个之前的列表，`start` 除外。

示例 例如：

```
\begin{enumerate}[resume*, start=1] % 或 [start=1, resume*]
```

使用前一个 `enumerate` 中的键，但重启计数器。

3.4 系列

`series=`⟨*series-name*⟩
⟨*series-name*⟩　`resume*=`⟨*series-name*⟩　`resume=`⟨*series-name*⟩

继续列表的另一种方法是使用键 `series`，使它们表现为一个单元。带键 `series` 的列表被视为起始列表，其设置全局存储，以便后续通过 `resume`/`resume*` 使用。所有这些键都接受系列名作为值（必须不同于现有键）：

- `resume=`⟨*series-name*⟩ 仅继续系列中的条目编号，
- `resume*=`⟨*series*⟩ 还应用起始列表的设置，
- ⟨*series*⟩，即系列名作为键，是 `resume*=`⟨*series*⟩ 的替代写法。

示例 考虑：

```
\begin{enumerate}[label=\arabic*(a), leftmargin=1cm, series=l_after]
\item A
\item B
\end{enumerate}
```

得到：1(a) 2(a)。可以继续：

```
\begin{enumerate}[label=\arabic*(b), resume*=l_after]
                  % 或 [label=\arabic*(b), l_after]
\item A
\item B
\end{enumerate}
```

得到：3(b) 4(b)。（但如果你愿意，也可以用 `start=1`。）

注意你可以添加更多参数，它们在起始列表保存的参数之后执行，因此优先于后者——特别是，`resume*` 本身优先于起始列表中的 `start`（例如 `start=1`）。

注意 每次启动一个系列时，内部会定义若干命令。因此，为避免浪费资源，不重叠的系列应使用相同名称。

警告 宏包未来可能引入新键，因此直接将 ⟨*series*⟩ 作为键是前向不兼容的潜在来源。然而，在键名中使用连字符或星号以外的非字母字符（例如`!notes` 或 `m_steps`）以及大写字母和数字是安全的，因为 enumitem 永远不会使用它们。 新特性 3.7 如果你定义了一些全小写名称的系列，而引入了新的冲突键，除了更改名称外，另一个选择是宏包选项 `series=override`（错误消息是 `Invalid series name `key'`）。使用它系列名称优先于预定义键——但仅在绝对必要时使用。

3.5 惩罚值

`beginpenalty=`⟨*integer*⟩ `midpenalty=`⟨*integer*⟩ `endpenalty=`⟨*integer*⟩

分别设置列表开始处、条目之间和列表结束处的惩罚值。请参阅 LaTeX 或 TeX 手册了解惩罚值如何控制分页。与其他参数不同，列表开始时它们的值不会重置为默认值，因此会应用于子列表。

3.6 注入代码

`before=`⟨*code*⟩ `before*=`⟨*code*⟩

在列表开始前执行代码（更准确地说，在用于定义列表的 `list` 环境的第二个参数中）。非星号形式设置要执行的代码，覆盖任何先前值；星号形式将代码添加到现有代码中（在设置层级中，见下文，不相对于外层列表/文本）。它可以包含例如规则和文本，但未经广泛测试。所有计算已完成，你可以访问和操作列表参数。

示例 将两边距（左右）都设为最宽标签宽度：

```
\setlist{leftmargin=*, before=\setlength{\rightmargin}{\leftmargin}}
```

`after=`⟨*code*⟩ `after*=`⟨*code*⟩

同上，但在列表即将结束前执行。

`first=`⟨*code*⟩ `first*=`⟨*code*⟩

新特性 3.6 同上，但作为列表正文中的第一件事，因此

```
\begin{itemize}[first=⟨code⟩]
```

等价于

```
\begin{itemize}
⟨code⟩
```

示例 通过 `first` 你可以为显示元素定义自己的环境。一个简单的例子：

```
\newlist{letter}{itemize}{1}
\setlist[letter]{first=\item[]\itshape, rightmargin=\leftmargin}
```

这里不需要 label，因为它不被使用。

`\EnumitemId`

新特性 3.7 为帮助某些任务，每个列表被分配一个唯一数字标识符，由\EnumitemId 返回。

示例 以下示例展示如何将\label 与\EnumitemId 结合使用，并用 after 自动将列表标签宽度设为最宽者（前提是 ref 等价于标签）：[4]

```
\SetEnumitemKey{widestlabel}
  {labelwidth = \widthof{\ref{enum-\EnumitemId}},
   after      = \label{enum-\EnumitemId}}
```

然后只需使用键 widestlabel。

示例 倒计数也是可行的，但稍微复杂一些，我们需要一些" 外部" 帮助。以下是一种可能的解决方案，但不是唯一的一种（很可能甚至不是最好的——例如，start 实际上是空操作）。

```
\newcounter{rev-enumi}   \newcounter{rev-enumii}
\newcounter{rev-enumiii} \newcounter{rev-enumiv}

\newcommand\revcounter[1]{%
  \setcounter{#1}{\value{#1}-2}%
  \stepcounter{rev-#1}%
  \gdef\afterrev{%
    \refstepcounter{rev-#1}%
    \setcounter{rev-#1}{0}}}

\AddEnumerateCounter\revcounter\revcounter{} % 第二个是占位符

\SetEnumitemKey{revRoman}
  {start  = \getrefnumber{enum-\EnumitemId}-1,
   label  = (\Roman*)\revcounter*,
   ref    = (\Roman*),
   after  = \afterrev
            \label{enum-\EnumitemId}}
```

3.7 描述样式

description 中可用的一个键。

`style=⟨name⟩`

设置描述样式。⟨*name*⟩ 可为以下之一：

standard 类似标准类中的 description，尽管在其他类中可能略有不同。标签被 boxed。设置 itemindent=!。

unboxed 很像标准 description，但标签不被 boxed，以避免不均匀间距和过长标签无法换行。设置 itemindent=!。

nextline 如果标签不适合边距，文本在下一行继续，否则放在宽度为\leftmargin − \labelsep 的框中，即条目正文永远不会伸入左边距。设置 labelwidth=!。

sameline 类似 nextline，但如果标签不适合边距，文本在同一行继续。等同于 style=unboxed,labelwidth=!。

[4] 参见 https://tex.stackexchange.com/questions/29322/how-to-make-enumerate-items-align-at-left-margin.

multiline 标签放在宽度为 leftmargin 的 parbox 中，必要时多行。等同于 style=standard,align=parleft,labelwidth=!。如果你修改它，记住 align 不能自由设置（出于内部优化）；特别是，水平框没有意义且可能引发错误。

警告

1. 混合使用 boxed 和 unboxed 标签没有明确定义的行为。
2. 嵌套列表时所有组合都允许，但并非所有都有意义。
3. 不支持嵌套 nextline 列表（可以工作，但未来行为可能改变，因为当前行为并不符合预期）。

3.8 紧凑列表

`noitemsep` `nosep`

键 noitemsep 消除条目和段落之间的间距（即 itemsep=0pt 和 parsep=0pt），而 nosep 消除所有垂直间距。[5]

3.9 "Wide" lists

`wide`
`wide=`⟨*parindent*⟩

使用这个便捷键，左边距为零，标签是文本的一部分——换句话说，条目看起来像普通段落。[6] 这里 labelsep 设置标签与第一个词之间的间距。等价于 equivalent to

```
align=left, leftmargin=0pt, labelindent=\parindent,
listparindent=\parindent, labelwidth=0pt, itemindent=!
```

使用 wide=⟨*parindent*⟩ 可以立即设置\parindent 以外的值。当然，这些键也可以在 wide 之后覆盖；例如，记住左对齐标签如果比 labelwidth 宽会推动文本，你可以设置 labelwidth=1.5em 作为最小宽度，或者你可能更喜欢 itemindent=* 而不是 itemindent=!，后者将最小宽度设为最宽标签的宽度。在第 2 层你可能更喜欢 labelindent=2\parindent，依此类推。你可能还想将其与 noitemsep 或 nosep 组合使用。

3.10 类似 enumerate 的标签

`shortlabels`（宏包选项）

使用宏包选项 shortlabels 你可以使用类似 enumerate 的语法，其中 A、a、I、i 和 1 分别代表\Alph* 、\alph*、\Roman*、\roman* 和\arabic* 。这主要是作为与 enumerate 宏包的一种兼容模式，因此适用以下特殊规则：如果（任意层级的）第一个选项不被识别为有效键，则被视为使用类似 enumerate 语法的标签。例如：

```
\begin{enumerate}[i), labelindent=\parindent, labelsep=*]
...
\end{enumerate}
```

如果 ref 与 label 不同，你可能还需要设置它。
虽然可能不那么有用，在类似条件下你也可以在 itemize 环境中省略 label=：

```
\begin{itemize}[\textbullet]
...
\end{itemize}
```

`\SetEnumerateShortLabel{`⟨*key*⟩`}{`⟨*replacement*⟩`}`

使用此命令，你可以定义新键（或重新定义），这对于将 enumerate 适配到特定排版规则或扩展到非拉丁脚本特别有用。这里 ⟨*replacement*⟩ 包含计数器的星号版本之一。

[5] 键 nolistsep 现已弃用，它引入了少量伸展，并非预期行为。
[6] fullwidth 已弃用。

示例 例如：

```
\SetEnumerateShortLabel{i}{\textsc{\roman*}}
```

重新定义 i，使使用此键的条目以小型大写罗马数字编号。

注意 键必须是单个字符。

3.11 通用键与值

`\SetEnumitemKey{`⟨*key*⟩`}{`⟨*replacement*⟩`}`

使用此命令你可以创建自己的（无值）键。如此定义的键可以像其他键一样使用。

示例 通过

```
\SetEnumitemKey{midsep}{topsep=3pt, partopsep=0pt}
```

你可以写

```
\begin{enumerate}[midsep]
```

示例 另一个例子是使用 multicol 的多列列表：

```
\SetEnumitemKey{twocol}{
  itemsep = 1\itemsep,
  parsep  = 1\parsep,
  before  = \raggedcolumns\begin{multicols}{2},
  after   = \end{multicols}}
```

这里，itemsep 和 parsep 的设置消除了伸展和收缩部分，在这种情况下改善了结果。当然，你可能需要定义一个新列表。

警告 本宏包未来可能引入新键，因此\SetEnumitemKey 存在前向不兼容的风险。但在键名中使用连字符或星号以外的非字母字符（例如:name 或 2_col）以及大写字母和数字是安全的，因为 enumitem 永远不会使用它们。

`\SetEnumitemValue{`⟨*key*⟩`}{`⟨*string-value*⟩`}{`⟨*replacement*⟩`}`

此命令为 ⟨*key*⟩=⟨*value*⟩ 对提供了进一步的抽象层。使用它你可以定义逻辑名称，它们被翻译为实际值。例如：

```
\SetEnumitemValue{label}{numeric}{\arabic*.}
\SetEnumitemValue{leftmargin}{standard}{\parindent}
```

你可以这样写：

```
\begin{enumerate}[label=numeric, leftmargin=standard]
```

因此，你可以将 label=numeric 的含义留到最终设计时决定。

4 行内列表

行内列表是在段落中作为普通文本设置的”水平”列表。使用本宏包你可以创建行内列表，如下所述，通过\newlist ，它们有自己的标签和计数器。然而，通常使用标准列表的行内版本、采用相同标签模式就足够了——宏包选项 inline 就是做这个的。

警告 条目被 boxed，因此浮动体丢失且不允许嵌套列表（记住许多显示元素被定义为列表）。链接可能指向错误位置，禁止使用行间公式，而且由于 TEX 构建列表时做的优化，显式断词可能错误。[7] 这个默认设置是有原因的，即此功能主要为短条目（几个词）设计，参数 `itemjoin*` 可用于逻辑标记。要克服这些限制，你可以设置 `mode=unboxed`，见下文。

`inline` (package option)
`enumerate*` `itemize*` `description*` (environments)

使用宏包选项 `inline`，三个行内列表环境被定义：`enumerate*`、`itemize*` 和 `description*`。它们模拟 paralist 和 shortlst 的行为，即标签和设置与显示（即正常）列表 `enumerate`、`itemize` 和 `description` 分别共享（但记住恢复基于环境名称，而非列表类型）。这仅适用于用 `inline` 创建的那些——用`\newlist` 创建的行内列表（如下所述）是独立的，使用自己的标签和设置。

注意 注意如果你不需要标准列表的行内版本，则不需要 `inline`，而是用`\newlist` 定义你自己的独立行内列表。

警告 由 `inline` 定义的这三个环境的设置与显示变体共享，因此不能用`\newlist` 直接重定义。尝试这样做会引发晦涩错误。如果你需要独立设置，用`\newlist` 定义而不用 `inline`。

`itemjoin=`⟨*string*⟩ `itemjoin*=`⟨*string*⟩ `afterlabel=`⟨*string*⟩

格式通过键 `itemjoin`（默认是空格）和 `afterlabel`（默认是`\nobreakspace` ，即 `~`）设置。另一个键是 `itemjoin*`，如果设置，它在最后一个条目之前替代 `itemjoin`。

`itemjoin` 在垂直模式下被忽略（即在 unboxed 模式下，以及在引用、显示列表等之后）。

示例 通过

```
before=\unskip{: }, itemjoin={{; }}, itemjoin*={{, and }}
```

条目之间使用以下标点：

Blah blah: (a) one; (b) two; (c) three, and (d) four. Blah blah

`mode=unboxed` `mode=boxed`

如果在行内列表中使用浮动体、列表或行间公式很重要，请使用替代” 模式”，通过 `mode=unboxed` 激活（默认是 `mode=boxed`）。使用它，浮动体可以自由使用，但错误放置的`\item` 不会被捕获，且 `itemjoin*` 被忽略（日志中会写入关于此事实的警告）。

5 全局设置

也可以进行应用于所有这些列表的全局更改：

`\setlist[enumerate,`⟨*levels*⟩`]{`⟨*format*⟩`}`
`\setlist[itemize,`⟨*levels*⟩`]{`⟨*format*⟩`}`
`\setlist[description,`⟨*levels*⟩`]{`⟨*format*⟩`}`
`\setlist[`⟨*levels*⟩`]{`⟨*format*⟩`}`

其中 ⟨*level*⟩ 是 `list` 中的列表层级（一个或多个），以及 `enumerate` 和 `itemize` 中的对应层级。[8] 不带 ⟨*levels*⟩ 时，格式应用于所有层级。这里的 list 不是指任意列表，而是仅指本宏包处理的三个列表，以及被本宏包重定义或用`\newlist` 定义的列表（见下文）。例如：

[7] Knuth 式的” 过早优化”？ 谁知道，但无论如何使用 LuaTEX 断词应该是正确的。
[8] `\setenumerate`、`\setitemize` 和`\setdescription` 已弃用。

```
\setlist{noitemsep}
\setlist[1]{labelindent=\parindent} % < Usually a good idea
\setlist[itemize]{leftmargin=*}
\setlist[itemize,1]{label=$\triangleleft$}
\setlist[enumerate]{labelsep=*, leftmargin=1.5pc}
\setlist[enumerate,1]{label = \arabic*.,
                      ref   = \arabic*}
\setlist[enumerate,2]{label = \emph{\alph*}),
                      ref   = \theenumi.\emph{\alph*}}
\setlist[enumerate,3]{label = \roman*),
                      ref   = \theenumii.\roman*}
\setlist[description]{font=\sffamily\bfseries}
```

这些设置按以下顺序读取：list、当前层级的 list、enumerate/itemize/description、以及当前层级的 enumerate/itemize/description；如果一个键多次出现且值不同，最后一个即最具体的值生效。如果我们用 `resume*` 恢复系列或列表，保存的键随后应用。最后，应用可选参数（`resume*` 除外）。

LaTeX 提供了一组宏来更改这些参数中的许多，但用本宏包设置它们更一致，有时更灵活，代价是更" 显式"（和冗长）。列表规范可以包含变量和计数器，前提是它们可展开，且计数器支持 calc，因此如果你加载本宏包，可以写：

```
\newcount{toplist}
\setcount{toplist}{1}
\newcommand{\mylistname}{enumerate}
\setlist[\mylistname,\value{toplist}+1]{labelsep=\itemindent+2em]
```

这允许借助循环定义列表。

警告 似乎没有办法捕获`\setlist` 或`\newlist` 中拼写错误的名称，会引发无意义的错误" 缺少数字，按零处理"。

6 依赖字号的设置

新特性 3.6 对于依赖字号的设置，大多数情况下你只需要 `ex` 或 `em` 等相对单位。有时你可能想要离散的步骤，特殊语法允许这样做。

使用此功能需要以下宏包选项。

`sizes`（宏包选项）

长度可以包含基于字号的设置，如下所示（第一个 < 之前的值是默认值）。

```
\setlist{
  topsep      = 20pt <-10> 6pt          <10-> 40pt,
  leftmargin  =      <-10> 0cm <10> 1cm <10-> 2cm ,
  rightmargin =      <-10> 0cm <10> 1cm <10-> 2cm ,
  }
```

也接受名称：`script`、`tiny`、`footnote`、`small`、`normal`、`large`、`Large`、`LARGE`、`huge`、`Huge`（即必要时从 LaTeX 名称中去掉 size）。例如：

```
\setlist{
  topsep      = 20pt <-normal> 6pt              <normal-> 40pt,
  leftmargin  =      <-normal> 0cm <normal> 1cm <normal-> 2cm ,
  rightmargin =      <-normal> 0cm <normal> 1cm <normal-> 2cm ,
  }
```

单值优先于范围（即特定值优先于通用值）。在范围内，最后一个匹配生效。范围 `a-b` 表示

$$a \leq \text{size} < b$$

（包含下限，不包含上限）。这些规则在上面的示例中允许在逻辑位置设置 10 或 normal。记住标称字号并不总是实际字号——例如，使用选项 11pt 时，\normalsize（和 normal）是 10.95。你可以在一个值前加多个单限定符，如 <small><normal>12pt。[9] 第一个 <..> 之前的值被视为默认值。

注意 出于效率原因，命名字号仅在加载 enumitem 时分配一次，假设它们由文档类设置，或先前加载的本地样式设置。

`\SetEnumitemSize{⟨name⟩}{⟨selector⟩}`

新特性 3.7 如果在加载 enumitem 后修改了字号，或者你使用的文档类有非标准字号（甚至你只是想要其他名称），可以用以下工具设置或重置。

示例 一个简单例子：

```
\SetEnumitemSize{normal}{\normalsize}
\SetEnumitemSize{large}{\large}
```

`\setlist<⟨size⟩>[⟨names⟩,⟨levels⟩]{⟨keys/values⟩}`

新特性 3.7 下面介绍的\setlist 扩展，将字号优先级规则也应用于此（因此\setlist 的顺序很重要），通过此过程定义的依赖字号的键优先于其他键。例如：

```
\setlist<-normal>[enumerate]{nosep}
```

但每个\setlist 中只接受一个字号限定符。

7 克隆基本列表

`\newlist{⟨name⟩}{⟨type⟩}{⟨max-depth⟩}`
`\renewlist{⟨name⟩}{⟨type⟩}{⟨max-depth⟩}`

三个列表可以被克隆，以便你定义表现类似它们的逻辑环境。要定义新列表（或重定义现有列表），使用\newlist （或\renewlist ），其中 ⟨type⟩ 是 enumerate、itemize 或 description。行内列表的类型为 enumerate*、itemize* 和 description*。

注意 行内 ⟨type⟩ 始终可用，即使没有宏包选项 inline，后者只是用这些名称定义对应类型的三个环境。

如果 ⟨type⟩ 是 enumerate，会定义一组名称为 ⟨name⟩i、⟨name⟩ii、⟨name⟩iii、⟨name⟩iv 等的计数器（取决于 ⟨max-depth⟩）。然后你可以在标签中使用这些计数器；例如，如果你定义了名为 steps 的列表，可以这样定义标签：

```
label=\arabic{stepsii}.\arabic{stepsi}
```

警告 不要为 ⟨max-depth⟩ 使用任意大的数字，以避免创建过多计数器和相关宏。

警告 为与 LaTeX 中的计数器命名方案保持一致，列表层级在内部也用罗马数字命名，即 ⟨list⟩i、⟨list⟩ii、⟨list⟩iii 等。由于这个原因（计数器和列表名称都是），定义两个列表如 books 和 booksi 会导致意外结果（目前没有任何警告，应该修复）。

`\setlist[⟨names⟩,⟨levels⟩]{⟨keys/values⟩}`
`\setlist*[⟨names⟩,⟨levels⟩]{⟨keys/values⟩}`

创建列表后，你可以（实际上必须，至少设置标签）用\setlist 设置新列表：

[9] 注意此语法紧密遵循 \DeclareFontShape，除了单值的优先级。

```
\newlist{ingredients}{itemize}{1}
\setlist[ingredients]{label=\textbullet}
\newlist{steps}{enumerate}{2}
\setlist[steps,1,2]{label=(\arabic*)}
```

`\setlist` 可选参数中的名称指定设置应用于哪些列表，数字指定层级（支持 `calc`）。可以给出多个列表和/或多个层级，所有组合都会被设置；例如：

```
\setlist[enumerate,itemize,2,3]{...}
```

不带数字（或 0）表示所有层级，不带名称表示所有列表；不带可选参数表示所有列表的所有层级。星号形式`\setlist*` 将设置添加到先前的设置中。你可以将添加限制在特定字号，如第 6 节所述。必须注意后者优先级高于星号形式（即为某些字号添加的设置优先于为所有字号添加的设置，因此应用键的最具体值）。

`\setlistdepth{`⟨*integer*⟩`}`

默认情况下，LaTeX 限制 5 个嵌套层级，但克隆列表时这个值可能太短，因此你可能想设置新值。在第 5 层以下（或文档类定义的最深层级以下），使用最后一层的设置（即`\@listvi` ）。

8 计数器详解

8.1 新计数器表示

`\AddEnumerateCounter{`⟨*LaTeX command*⟩`}{`⟨*internal command*⟩`}{`⟨*widest label*⟩`}`

”注册”一个计数器表示，使 enumitem 识别它。主要用于非拉丁文字，但在拉丁文字中也有用。

示例 以下示例定义了一个带有序数名称的新计数器：

```
\makeatletter
\def\ctext#1{\expandafter\@ctext\csname c@#1\endcsname}
\def\@ctext#1{\ifcase#1\or First\or Second\or Third\or
Fourth\or Fifth\or Sixth\fi}
\makeatother
\AddEnumerateCounter{\ctext}{\@ctext}{Second}
```

注意 计数器名称可以包含 `@`，即使它不是字母，也不会引发错误，如上例所示。

星号变体允许给出数字而不是字符串作为最宽标签。

示例 如果最宽标签对应值 2：

```
\AddEnumerateCounter*{\ctext}{\@ctmoreext}{2}
```

如果表示不是普通字符串而是带样式的（例如小型大写），首选此变体。

示例 俄语的例子：

```
\AddEnumerateCounter*{\asbuk}{\c@asbuk}{7}
```

8.2 重启 `enumerate`

`\restartlist{`⟨*list-name*⟩`}`

目前你可以通过以下方式获得文档中连续编号：

```
\setlist[enumerate]{resume}
```

\restartlist 用于在文档中途重启计数器。例如，你可以在章节开始时发出\restartlist ，使每章都有连续编号。

它仅基于列表名称，不基于列表类型，这意味着用宏包选项 inline 定义的 enumerate* 与 enumerate 不同，因为它的名称不同。

9 宏包选项

除了 inline、ignoredisplayed、sizes、series=override 和 shortlabels，还有以下选项。

loadonly

使用此宏包选项，宏包被加载但三个列表不被重定义。你可以创建自己的列表，甚至重定义现有的。

10 trivlist 问题

LaTeX 使用名为 trivlist 的 list 简化版本来设置显示元素，如 center、tabbing、theorem 等，即使概念上它们不是列表。不幸的是，trivlist 使用当前列表设置，这有一个奇怪的副作用：更改列表的垂直间距有时也会更改这些环境中的间距。本宏包修改了 trivlist，使当前层级的默认设置（即相应 clo 文件设置的那些）被重新设置。在标准 LaTeX 中这通常是多余的，但如果我们想微调列表，不重置默认值可能是个真正的问题（特别是如果你使用 nosep 选项）。垂直间距的最小控制可通过以下方式实现[10]

- \setlist[trivlist,⟨*level*⟩]{⟨*keys/values*⟩}

但 trivlist 本身（不常直接使用）不接受可选参数。此功能并非旨在作为完整的 trivlist 格式化工具。如果出于某种原因你不想更改 trivlist 并保留原始定义，可以使用宏包选项 ignoredisplayed。新特性 3.6 另一方面，如果你还想将所有列表的更改应用于 trivlist，只需设置宏包选项 includedisplayed。

11 示例

在这些示例中，我们设置\setlist{noitemsep}

```
在拉曼恰的一个地方，其名我不愿提起，
不久前住着一位绅士
\begin{enumerate}[labelindent=\parindent,leftmargin=*]
  \item 一支长矛，
  \item 一面旧盾，
  \item 一匹瘦马，
  \item 一只猎犬。
\end{enumerate}
一锅肉多菜少的炖菜，多数晚上吃凉拌肉，
星期六吃煎羊肉……
```

线显示 labelindent。

在拉曼恰的一个地方，其名我不愿提起，不久前住着一位绅士

1. 一支长矛，
2. 一面旧盾，
3. 一匹瘦马，
4. 一只猎犬。

一锅肉多菜少的炖菜，多数晚上吃凉拌肉，星期六吃煎羊肉……

[10] \setdisplayed 已弃用。

使用\begin{enumerate}[leftmargin=*] % labelindent 默认为 0pt。
线显示 labelindent。

在拉曼恰的一个地方，其名我不愿提起，不久前住着一位绅士

1. 一支长矛，
2. 一面旧盾，
3. 一匹瘦马，
4. 一只猎犬.

一锅肉多菜少的炖菜，多数晚上吃凉拌肉，星期六吃煎羊肉……

使用\begin{enumerate}[leftmargin=\parindent] 。
线显示 leftmargin。

在拉曼恰的一个地方，其名我不愿提起，不久前住着一位绅士

1. 一支长矛，
2. 一面旧盾，
3. 一匹瘦马，
4. 一只猎犬.

一锅肉多菜少的炖菜，多数晚上吃凉拌肉，星期六吃煎羊肉……

使用\begin{enumerate}[labelindent=\parindent, leftmargin=*, label=\Roman*., widest=III, align=left].
线显示 labelindent。Note

在拉曼恰的一个地方，其名我不愿提起，不久前住着一位绅士

I. 一支长矛，
II. 一面旧盾，
III. 一匹瘦马，
IV. 一只猎犬.

一锅肉多菜少的炖菜，多数晚上吃凉拌肉，星期六吃煎羊肉……

使用\begin{enumerate}[label=\fbox{\arabic*}] 。第一个条目为 1

在拉曼恰的一个地方，其名我不愿提起，不久前住着一位绅士

1 一支长矛，
2 一面旧盾，
3 一匹瘦马，
4 一只猎犬.

一锅肉多菜少的炖菜，多数晚上吃凉拌肉，星期六吃煎羊肉……

嵌套列表。

```
在拉曼恰的一个地方，其名我不愿提起，
不久前住着一位绅士
\begin{enumerate}[label=(\alph*), labelindent=\parindent,
    leftmargin=*, start=12]
\item 一支长矛，
\begin{enumerate}[label=(\alph{enumi}.\roman*), leftmargin=*, start=7]
\item 一面旧盾，
\end{enumerate}
\item 一匹瘦马，
\begin{enumerate}[label=(\alph{enumi}.\roman*), leftmargin=*, resume]
\item 一只猎犬。
\end{enumerate}
\end{enumerate}
一锅肉多菜少的炖菜，多数晚上吃凉拌肉，
星期六吃煎羊肉……
```

在拉曼恰的一个地方，其名我不愿提起，不久前住着一位绅士

(l) 一支长矛，

(l.vii) 一面旧盾，

(m) 一匹瘦马，

(m.viii) 一只猎犬.

一锅肉多菜少的炖菜，多数晚上吃凉拌肉，星期六吃煎羊肉……

```
在拉曼恰的一个地方，其名我不愿提起，
不久前住着一位绅士
\begin{description}[font=\sffamily\bfseries, leftmargin=3cm,
    style=nextline]
  \item[\textnormal{堂吉诃德最先有的}] 一支长矛，
  \item[\textnormal{其次}] 一面旧盾，
  \item[\textnormal{再次}] 一匹瘦马，
  \item[\textnormal{最后，第四样}] 一只猎犬。
\end{description}
一锅肉多菜少的炖菜，多数晚上吃凉拌肉，
星期六吃煎羊肉……
```

在拉曼恰的一个地方，其名我不愿提起，不久前住着一位绅士

堂吉诃德最先有的 一支长矛，
其 次 一面旧盾，
再 次 一匹瘦马，
最 后， 第 四 样 一只猎犬.

一锅肉多菜少的炖菜，多数晚上吃凉拌肉，星期六吃煎羊肉……

同上，但使用`sameline`。

在拉曼恰的一个地方，其名我不愿提起，不久前住着一位绅士

堂吉诃德最先有的 一支长矛，
其 次 一面旧盾，
再 次 一匹瘦马，
最 后， 第 四 样 一只猎犬.

一锅肉多菜少的炖菜，多数晚上吃凉拌肉，星期六吃煎羊肉……

同上，但使用`multiline`。注意如果条目正文太短。

在拉曼恰的一个地方，其名我不愿提起，不久前住着一位绅士

堂吉诃德最先有的 一支长矛，
其 次 一面旧盾，
再 次 一匹瘦马，
最 后， 第 四 样 一只猎犬.

一锅肉多菜少的炖菜，多数晚上吃凉拌肉，星期六吃煎羊肉……

12 后记

12.1 LaTeX 列表

众所周知，LaTeX 预定义了三个列表：`enumerate`、`itemize` 和 `description`。这是一种非常常见的分类，在 HTML 中也能找到。然而，存在一个更通用的模型，基于三个字段——即标签、标题和正文——因此 enumerate 和 itemize 有标签（编号和不编号）但没有标题，而 description 有标题但没有标签。在这个模型中，你可以有一个带标签条目的 description，例如（当然，这个简单的方案远不能令人满意）：

```
\newcommand\litem[1]{\item{\bfseries #1,\enspace}}
\begin{itemize}[label=\textbullet]
\litem{堂吉诃德最先有的} 一支长矛，
……等等。
```

在拉曼恰的一个地方，其名我不愿提起，不久前住着一位绅士

- **堂吉诃德最先有的**，一支长矛，
- **其次**，一面旧盾，
- **再次**，一匹瘦马，
- **最后，第四样**，一只猎犬.

12.2 已知问题

- 列表恢复基于环境名称，当\newenvironment 包含列表时，你可能想使用\begin{⟨*list*⟩} 和\end{⟨*list*⟩}。使用相应的低级命令（即\⟨*list*⟩ 和\end⟨*list*⟩）不是错误，但确保结果正确是你的责任。
- boxed 标签（包括 enumerate 和 itemize）与 unboxed 标签混合使用的行为没有明确定义。boxed 和 unboxed 行内列表也是如此（甚至可能引发错误）。
- 类似地，同时恢复系列和列表也是允许的，但同样其行为没有明确定义。
- (3.5.2) 出现了与 2.x 的不兼容——如果你使用可选参数向\ref 或其他需要可展开宏的宏传递值，会引发错误。快速修复是在 before 中让\makelabel 指向\descriptionlabel。

12.3 3.0 新特性

- 行内列表，带设置条目如何连接的键（即条目间的标点）。提供两种模式：boxed 和unboxed。
- \setlist 支持 calc（例如在循环中使用），你可以同时设置不同列表和层级。
- 所有与标签相关的长度都可以取* 值（不仅限于labelsep 和leftmargin）。其行为已保持一致，并有新值!，它不计算最宽标签。
- 使用\restartlist{⟨*list-name*⟩}，可以重启列表计数器（在使用resume 的情况下）。
- resume* 可以与其他键组合使用。
- 列表可以用系列全局收集，使它们被视为一个列表。要启动系列只需使用series=⟨*series-name*⟩，然后用resume=⟨*series-name*⟩ 或resume*=⟨*series-name*⟩ 恢复。
- " 实验性的" fullwidth 已被新键wide 取代。
- \SetLabelAlign 定义新的对齐值。
- 你可以定义" 抽象" 值（例如label=numeric）和新键。

- (3.2) start 和widest* 支持 calc。
- (3.2) \value 可与widest* 一起使用。
- (3.2) 移除了\arabic 等的一些内部限制。更灵活，代价是错误检查更" 宽松"。

12.4 错误修复

- 星号值（例如leftmargin=*）不能被覆盖，新值被忽略。
- nolistsep 作为多个键中的第一个时不总是被识别，因此被当作短标签处理（即nol\roman*stsep）。
- labelwidth 不总是工作（当先前有widest 和* 时）。
- 使用align=right 时标签和后续文本可能重叠。
- description 没有获得正确的列表层级。
- 在某个版本（2.x?）\value* 停止工作。
- (3.1) 不幸的是，xkeyval " 杀死" 了 keyval，因此后者已在 enumitem 中复制。
- (3.3) 修复了一个严重错误——使用* 时itemize 和description 都不工作。
- (3.4) 修复了 boxed 模式中的间距问题（第一个条目前\unskip 位置错误，条目间 spacefactor 错误）。
- (3.4) nolistsep 未按预期工作，但由于这个错误已存在多年，提供了新键nosep。
- (3.4) nolistsep 与shortlabels 的问题（见上文）并非在所有情况下都修复。希望现在修复了。
- (3.5.0) 修复了与条目间 spacefactor 相关的问题。
- (3.5.0) 修复了嵌套 boxed 行内列表的问题。
- (3.5.1) resume* 只工作一次，后续的行为像 resume。
- (3.5.2) 修复了不工作的\setlist*。

12.5 致谢

我特别感谢 Lars Madsen 的评论和建议，他还发现了一些错误。